Universidad Tecnológica Nacional
Facultad Regional Córdoba
Ing. en Sistemas de Información

EXAMEN PARCIAL 4

Algoritmos y Estructuras de Datos

Fecha: 18/10/2025

Turno T2 - Enunciado:

La Academia de Hechicería Howarts necesita un programa para registrar a sus estudiantes y evaluar su progreso en el dominio de la magia. Cada año se realiza un informe general para determinar cuáles alumnos están listos para avanzar de nivel o ser considerados para participar en torneos mágicos intercolegiales. Por cada *Mago* se conoce: nombre (cadena de caracteres), casa (entero entre 1 y 4, donde: 1–Gryffindor, 2–Hufflepuff, 3–Slytherin, 4–Ravenclaw), nivel mágico (entero entre 1 y 10, donde 1 es *Aprendiz* y 10 es *Mago Supremo*), cantidad de hechizos dominados (entero positivo), y puntaje general (entero positivo).

En base a lo anterior, se provee **ya desarrollado**:

- ✓ Un módulo *clase.py* que contiene la definición de la clase **Mago** con el *método constructor* y el método *__str__()* ya incluidos. NO CAMBIE NADA. NO AGREGUE NADA. DEJE LA CLASE TAL CUAL ESTÁ.
- ✓ Un segundo módulo *principal.py* que contiene una función *main()* con el esquema del menú de opciones ya desarrollado y funcionando, de forma que esa función *main()* sea usada como la función de entrada del programa completo y una función de validación de carga. En el mismo módulo, se provee ya desarrollada una función *generar_aleatorio()* que crea con valores aleatorios un objeto de la clase **Mago** y lo retorna ya listo para usar. Los procesos para cumplir con cada opción del menú, son justamente la tarea que debe programar cada estudiante. Las opciones a programar son:

---

1. Cargar un arreglo de registros con los datos de *n* magos (cargar *n* por teclado y validar que sea positivo), de manera que en todo momento el arreglo se mantenga ordenado por **nombre** del mago. Para esto debe utilizar el algoritmo de inserción ordenada con búsqueda binaria (se considerará directamente incorrecta la solución basada en cargar el arreglo completo y ordenarlo al final, o aplicar el algoritmo de inserción ordenada, pero con búsqueda secuencial). La carga de los magos debe ser automática y con base aleatoria (use la función *generar_aleatorio()* que ya tiene disponible en el módulo *principal.py* para crear los objetos). Cada vez que se ingrese en esta opción, el arreglo debe ser creado desde cero y todo contenido anterior debe ser eliminado.

   Ajuste de runner: al ejecutar esta opción, solo cargue por teclado el valor de n, y no muestre nada al terminar de cargar el vector. Solo regrese al menú.

---

2. Mostrar todos los estudiantes del arreglo, a razón de un registro por línea, y contar y mostrar cuántos pertenecen a la casa *c1* (que se carga por teclado).

   Ajuste de runner: al ejecutar esta opción, cargue por teclado el valor de *c1*, muestre el listado pedido usando el método __str__() que ya tiene en la clase, muestre al final el promedio pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

---

3. Buscar si existe un mago cuyo nombre sea *nom* (la variable *nom* se ingresa por teclado). Si existe, mostrar todos sus datos. Si además su nivel mágico es 5 o menos, incrementar su puntaje general en un 14%, y volver a mostrar sus datos completos. Si el mago no existe, informar con el mensaje “¡Mago inexistente!”.

   Ajuste de runner: al ejecutar esta opción, cargue por teclado el nombre en la variable *nom*, muestre el registro (si existe) o el mensaje "No existe" (si no existe), muestre en otra línea registro actualizado con el 14% (si corresponde) y regrese al menú (no muestre nada más que eso).

---

4. A partir del arreglo, generar un archivo binario donde se incluyan todos los magos cuyo puntaje general sea mayor o igual al puntaje promedio de todos los magos registrados.

   Ajuste de runner: al ejecutar esta opción, solo genere el archivo pedido y regrese al menú (no muestre nada en la pantalla al terminar).

5. Mostrar el contenido del archivo binario, a razón de un registro por línea en la pantalla, indicando además una línea extra que muestre el puntaje promedio de los estudiantes pertenecientes a la casa *Ravenclaw*.

   Ajuste de runner: al ejecutar esta opción, muestre el listado pedido usando el método __str__() que tiene disponible en la clase, muestre al final el promedio pedido aquí (use ":" para separar el mensaje de salida y el valor mostrado), y regrese al menú (no muestre nada más que eso).

Criterios generales de evaluación.

- Estructura completa del programa terminado **[máximo: 4 puntos por convenciones de estilo, la estructura de las funciones que desarrolle, el control de ejecución del módulo principal, todo otro elemento que aporte a la estructura del programa]**
- Desarrollo correcto del ítem 1: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 2: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 3: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 4: **[máximo: 4 puntos]**
- Desarrollo correcto del ítem 5: **[máximo: 4 puntos]**
- Para aprobar el parcial (con nota mínima de 4(cuatro), el alumno debe llegar a un *porcentaje de al menos 55% del puntaje máximo (eso es, un poco más de 13 puntos)*.